A

Project Report On

"Smart Factory IoT Dashboard With Equipment Control"

By

Sumedh Sanjay Varadkar (X-22-0445)

Shubham Rajan Malvankar (X-22-0503)

Shreyash Babaso Patil (X-22-0438)

Sanket Tatyaso Balanna (XD-23-0515)

Submitted in Partial Fulfillment of the requirements prescribed by

University of Mumbai for the **Major Project – II (SEMESTER VIII)**

in

Electronics and Telecommunication Engineering

Under the Guidance of

Prof. V. D. Chavan

Department of Electronics and Telecommunication Engineering

Finolex Academy of Management & Technology, Ratnagiri

Academic Year 2025-26

CERTIFICATE

Major Project - II Report Entitled

"Smart Factory IoT Dashboard With Equipment Control"

Duly completed by

Sumedh Sanjay Varadkar (X-22-0445)

Shubham Rajan Malvankar (X-22-0503)

Shreyash Babaso Patil (X-22-0438)

Sanket Tatayaso Balanna (XD-23-0515)

As a partial fulfillment of the Term work requirements for the Major Project - II (Eighth Semester) in Electronics and Telecommunication Engineering department From the ***University of Mumbai***

(Project Guide) **(Internal Examiner)** **(External Examiner)**

Dr. S. V. Chougule
HOD EXTC

Dr. Kaushal Prasad
Principal

Date: ……/ ….. /…….

Place: Department of Electronics & Telecommunication Engineering, FAMT, Ratnagiri.

DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Sumedh Sanjay Varadkar

Shubham Rajan Malvankar

Shreyash Babaso Patil

Sanket Tatayaso Balanna

Date:

Place:

ACKNOWLEDGEMENT

It gives us immense pleasure to present the report of our project here. It has been quite experience, facing a number of problems at stages and coming up with appropriate solutions, at the time the discussion amongst us or suggestions from our friends and teachers.

We thank our guide **Prof. V. D. Chavan, Assistant Professor,** Department of Electronics and Telecommunication Engineering for constantly guiding and motivating us.

It is our humble request to thank all those who contributed towards the successful completion of this project, especially our institute and professors who have helped us in achieving the desired success in this project by making available to us the tools and means to complete it.

We express our gratitude to **Dr. Sharada V. Chougule, Professor and Head of the Department**, Electronics and Telecommunication Engineering for her invaluable suggestions and constant encouragement.

Lastly, we would like to put our thanks on record to the teaching and non-teaching staff for rendering their support directly or indirectly.

ABSTRACT

The Smart Factory IoT Dashboard with Equipment Control is designed to provide a real-time monitoring and control system for industrial environments using IoT technology. In this project, the ESP32 microcontroller is used to collect data from sensors such as temperature, humidity, and object detection sensors. This data is transmitted over Wi-Fi to the Firebase Realtime Database, where it is stored and continuously updated. A web-based dashboard is developed to display this data in an easy-to-understand format, allowing users to monitor factory conditions from anywhere.

In addition to monitoring, the system also enables remote control of industrial equipment such as fans, lights, conveyors, and buzzers through a 4-channel relay module. Control commands given through the dashboard are sent to Firebase and then executed by the ESP32 in real time. This project helps in reducing manual effort, improving efficiency, and minimizing the chances of equipment failure. Overall, the system demonstrates a practical implementation of IoT in industrial automation, supporting the concept of smart manufacturing and Industry 4.0.

CONTENTS

***Abstract*……..**			**i**
List of Figures……..			**iv**
List of Appendix……..			**v**
List of Abbreviations……			**vi**
Chapter	**Chapter Name**		**Page No.**
1	**Introduction**		1
	1.1	Introduction……	2
	1.2	Motivation……..	3
	1.3	Problem Statement…….	4
	1.4	Objective of Project Work……..	4
	1.5	Organization of Project……..	5
		1.5.1 System Overview……...	5
		1.5.2 Hardware Architecture……...	6
		1.5.3 Software Architecture……...	6
		1.5.4 Communication Architecture……	7
		1.5.5 Cost Estimation…….	8
2	**Literature Review**		9-14

3	**Overview of System and Software Development**		15
	3.1	Overview of System…….	16
	3.2	Block Diagram of Smart Factory Iot Dashboard With Equipment Control……...	17
	3.3	Description of Block Diagram……...	17-19
	3.4	Working of Model…….	20-22
4	**Hardware and Software Implementation**		23
	4.1	Hardware Components…….	24-28
	4.2	Simulation……	29-30
	4.3	Programming Language and Software Used……	31-32
	4.4	Arduino IDE code Upload……	33
	4.5	Cloud Platform – Firebase Realtime Database……	34
	4.6	Web Dashboard……...	35-37
	4.7	System Dataflow……...	38
	4.8	Operating System ……..	39
	4.9	Flowchart……	40
5	**Results and Discussion**		41
	5.1	Hardware Implementation Output Images ……	42-43
	Future Scope		44
	Conclusion		45
	References		46
	Appendix – A		47-53

LIST OF FIGURES

Sr. No.	Title	Page No.
Fig 3.1	Block Diagram of Smart Factory IoT Dashboard With Equipment Control	17
Fig 4.1	ESP-32 DEV V1……	24
Fig 4.2	DC Motor (60 RPM)…….	24
Fig 4.3	DHT-11 Sensor…….……..	25
Fig 4.4	Buzzer …….…….	25
Fig 4.5	IR Sensor ……..……	26
Fig 4.6	4 Channel Relay Module…….	26
Fig 4.7	Single Relay ……	27
Fig 4.8	LED Bulb Zero Watt……..……..	27
Fig 4.9	Fan……..……..	28
Fig 4.10	DC Adapter ……..……	28
Fig 4.11	Wokwi Simulation ……...……	29
Fig 4.12	Embedded Programming Using Arduino IDE……...……	32
Fig 4.13	Arduino IDE Code Upload…….……..	33
Fig 4.14	Cloud Platform Firebase Realtime Database……	34
Fig 4.15	Login Page……	36
Fig 4.16	Home Page……...……	36
Fig 4.17	Control Dashboard..…….	36
Fig 4.18	About Page……..	37
Fig 4.19	Contact Us Page ……	37
Fig 4.20	System Dataflow…….	38
Fig 4.21	Flowchart ……	40
Fig 5.1	Smart Factory IoT Dashboard With Equipment Control …….	42
Fig 5.2	Web Dashboard ……...	42
Fig 5.3	Firebase Realtime Database ……	43

LIST OF APPENDIX

Appendix A	**Page No.**
Plagiarism Report ……	47
Research Paper Publication Certificate……..……..	48-52

Chapter 1

Introduction

1.1 Introduction:

In recent years, the rapid growth of the Internet of Things (IoT) has significantly transformed traditional industrial systems into smarter and more efficient environments. Industries are now moving towards automation and real-time monitoring to improve productivity, reduce human effort, and ensure better decision-making. The concept of smart factories, which is a key part of Industry 4.0, focuses on integrating sensors, communication technologies, and data analysis to create intelligent systems that can operate with minimal manual intervention.

The Smart Factory IoT Dashboard with Equipment Control is developed with the aim of providing a reliable and cost-effective solution for industrial monitoring and automation. In this project, an ESP32 microcontroller is used to collect real-time data from sensors such as temperature, humidity, and IR sensors. This data is transmitted to a cloud-based platform, Firebase Realtime Database, using Wi-Fi connectivity. A web-based dashboard is designed to display this information in a clear and user-friendly manner, allowing users to monitor various factory parameters remotely.

Along with monitoring, the system also enables remote control of industrial equipment such as fans, lights, conveyors, and buzzers using a relay module. Commands given through the dashboard are sent to the database and executed by the ESP32, ensuring smooth and real-time operation. This project not only reduces manual intervention but also enhances safety, efficiency, and reliability in industrial processes. It demonstrates how IoT technology can be effectively used to build smart and connected industrial systems.

1.2 Motivation:

In many small and medium-scale industries, monitoring and controlling equipment is still carried out manually, which often leads to delays, inefficiencies, and increased chances of human error. Operators are required to be physically present to check machine conditions such as temperature, humidity, and operational status. This not only consumes time but also makes it difficult to respond quickly to unexpected situations like overheating or equipment failure. These challenges highlight the need for a smarter and more reliable system.

The motivation behind this project comes from the growing demand for automation and real-time monitoring in industrial environments. With the advancement of IoT technologies, it has become possible to collect and transmit data instantly, enabling better decision-making and faster response. By integrating sensors, microcontrollers like ESP32, and cloud platforms such as Firebase, industries can achieve continuous monitoring and remote control without depending heavily on manual supervision. This inspired us to design a system that is both efficient and easy to implement.

Another key motivation was to develop a low-cost and scalable solution that can be adopted by industries without requiring complex infrastructure. Many existing industrial automation systems are expensive and difficult to maintain, which limits their use in smaller setups. Our project aims to bridge this gap by providing a simple yet effective IoT-based solution that improves productivity, enhances safety, and reduces operational costs. It also gave us an opportunity to gain practical knowledge and understand the real-world application of IoT in smart manufacturing.

.

1.3 Problem Statement :

Factories often face delays and breakdowns because machines are hard to monitor. A ESP32 – based IoT dashboard can show machine data and let users control them remotely, improving efficiency and reducing downtime.

1.4 Objective of Project Work:

- To design a smart factory monitoring system using the ESP32 microcontroller.
- To collect real-time data from industrial sensors such as temperature and humidity.
- To store and manage data using Firebase Realtime Database.
- To develop a web-based dashboard for real-time monitoring of factory parameters.
- To enable remote control of industrial equipment through Wi-Fi using the dashboard.
- To improve industrial safety, efficiency, and reduce manual intervention.
- To provide a low-cost, scalable, and IoT-based automation solution.

1.5 Organization of report:

This section provides a comprehensive look at the project's structure and execution. It begins with a system overview, offering a high-level understanding of the entire system. Following this, the report delves into the technical details, outlining the software architecture and hardware architecture. The communication architecture then clarifies how different components of the system interact.

1.5.1 System Overview : The proposed system is a Smart Factory IoT Dashboard designed to provide efficient monitoring and control of industrial equipment using modern IoT technology. In this system, the ESP32 microcontroller acts as the central unit that collects real-time data from sensors such as the DHT11 for temperature and humidity and an IR sensor for object or motion detection. This data is processed and transmitted to the Firebase Realtime Database through Wi-Fi connectivity, enabling continuous data storage and real-time updates. A user-friendly web-based dashboard is developed using HTML, CSS, and JavaScript to visually display the sensor data, equipment status, and system conditions from any remote location. Along with monitoring, the system also supports remote operation of industrial devices such as fans, lights, conveyors, and buzzers using a 4-channel relay module. Control commands provided through the dashboard are sent to Firebase and then fetched by the ESP32, which accordingly switches the devices ON or OFF. Overall, the system ensures better automation, reduces the need for manual intervention, improves operational efficiency, and provides a scalable solution for smart industrial environments.

1.5.2 Hardware Architecture : The hardware architecture of the proposed system is built around the ESP32 microcontroller, which acts as the central controller responsible for data processing, communication, and overall system coordination. It is interfaced with multiple sensors, including the DHT11 sensor for measuring temperature and humidity and an IR sensor for detecting objects or motion within the system. These sensors continuously send real-time data to the ESP32, which processes and prepares it for transmission. For controlling electrical devices, a 4-channel relay module is connected to the ESP32, allowing it to safely operate high-voltage equipment such as a fan, a zero-watt LED bulb, a DC motor used as a conveyor, and a buzzer for alerts. The relay module acts as a switching interface between low-power control signals from the ESP32 and high-power industrial devices. All components are powered using an appropriate DC power supply, ensuring stable operation of both the microcontroller and connected devices. Proper connections and interfacing are maintained to ensure reliable communication between sensors, controller, and actuators. The overall hardware design is kept simple, modular, and cost-effective, making it easy to implement, maintain, and expand for future industrial applications.

1.5.3 Software Architecture : The software architecture of the system is designed to ensure smooth interaction between the ESP32 microcontroller, the cloud database, and the web-based user interface. The ESP32 is programmed using Embedded C in the Arduino IDE, where it continuously reads data from sensors like the DHT11 and IR sensor, processes it, and sends it to the Firebase Realtime Database through Wi-Fi. Firebase acts as a central platform for real-time data storage and synchronization. A web dashboard developed using HTML, CSS, and JavaScript retrieves this data from Firebase and displays it to the user in real time. Additionally, when the user sends control commands through the dashboard, they are updated in Firebase and fetched by the ESP32, which then controls the connected devices via the relay module. This architecture enables efficient real-time monitoring and remote control with reliable communication between all system components.

1.5.4 Communication Architecture : The communication architecture of the proposed system is designed to enable reliable and real-time data exchange between the ESP32 microcontroller, the Firebase Realtime Database, and the web-based dashboard using Wi-Fi connectivity. The ESP32 connects to a wireless network and serves as the main communication gateway between the physical hardware and the cloud platform. It continuously collects data from sensors such as the DHT11 and IR sensor, processes the data, and transmits it to Firebase, where it is stored and updated in real time. Firebase acts as a centralized communication hub that ensures seamless synchronization between the ESP32 and the user interface. The web-based dashboard retrieves live data from Firebase and displays it to the user in a clear and interactive format. In addition to data monitoring, the system also supports two-way communication for control purposes. When a user sends commands through the dashboard, such as turning devices ON or OFF, these commands are updated in Firebase and instantly accessed by the ESP32. The ESP32 then processes these commands and controls the connected devices through the relay module. This continuous exchange of data and control signals ensures fast response, minimal delay, and efficient remote operation, making the system highly reliable and suitable for smart industrial applications.

1.5.5 Cost Estimation :

Sr. No	Components	Price
1	ESP32	Rs 550
2	DC Motor (60 rpm)	Rs 370
3	4-Channel Relay module	Rs 200
3	Led Bulb	Rs 80
4	Conveyor	Rs 2000
5	DHT-11 Sensor	Rs 70
6	IR Sensor	Rs 45
7	Buzzer	Rs 60
8	Fan	Rs 250
	Total Cost	Rs 3,625

Chapter 2

Literature Review

The rapid development of Internet of Things (IoT) technology has significantly transformed industrial systems by enabling real-time monitoring, automation, and data-driven decision-making. Traditional industrial environments often rely on manual supervision, which leads to inefficiencies, delayed fault detection, and increased operational costs. To overcome these challenges, researchers have proposed IoT-based smart monitoring systems that integrate sensors, microcontrollers, and cloud platforms. These systems allow continuous data collection and remote accessibility, thereby improving productivity and system reliability. The adoption of such technologies plays a key role in the evolution of Industry 4.0, where smart factories operate with minimal human intervention and enhanced efficiency.

The paper by M. S. Bakare and co-authors [1] presents an IoT-based indoor environmental monitoring system built around the ESP32 microcontroller and integrated with cloud services for real-time data access. The system collects key environmental parameters such as temperature, humidity, and air quality using connected sensors, and transmits this data over Wi-Fi to a cloud platform for storage, visualization, and analysis. A major focus of the study is on creating a low-cost, energy-efficient, and scalable solution that can be deployed in homes, offices, or industrial settings. The authors demonstrate how real-time monitoring and remote access through dashboards or mobile interfaces can help improve indoor comfort, detect abnormal conditions early, and support smarter decision-making for environmental control. Overall, the research highlights the practical benefits of combining ESP32-based hardware with cloud integration to build efficient smart monitoring systems.

The research conducted by S. Sharma (2026) [2] focuses on an intelligent low-cost IoT monitoring system using the Firebase cloud platform for real-time data management. The study demonstrates how ESP32-based devices can collect environmental parameters such as temperature, pH, and turbidity, and transmit the data to Firebase for storage and visualization through web dashboards. It also highlights the integration of lightweight machine learning techniques (TinyML) for on-device anomaly detection, improving system efficiency and reducing dependency on constant cloud connectivity. This research is relevant to our project as it supports the use of Firebase and ESP32 for building a scalable, cost-effective, and real-time industrial monitoring system [2].

The research conducted by Abdul Hasib and A. S. M. A. S. Akib (2026) [3] presents a cloud-enabled IoT system for real-time monitoring and control using the Firebase platform. The study demonstrates how IoT devices, integrated with microcontrollers like ESP32, can collect sensor data and transmit it to Firebase for real-time storage, visualization, and remote control through web or mobile interfaces. It also highlights the advantages of cloud-based architecture, such as scalability, low latency, and ease of integration for smart applications. This research is relevant to our project as it supports the use of Firebase cloud services for building an efficient, real-time industrial monitoring and control system [3].

The research conducted by Abdul Hasib and A. S. M. A. S. Akib (2026) [4] introduces *HydroSense*, an IoT-based framework for real-time monitoring with cloud integration. The study demonstrates how sensor data related to water or environmental parameters can be collected using microcontrollers like ESP32 and transmitted to cloud platforms for continuous monitoring and analysis. It highlights features such as real-time data visualization, remote accessibility, and efficient data handling through cloud services. The research also emphasizes scalability and cost-effectiveness, making the system suitable for smart environmental and industrial applications. This research is relevant to our project as it supports the use of cloud-integrated IoT frameworks for building efficient, real-time monitoring systems [4].

The research conducted by I. N. Afifah et al. (2026) [5] presents an ESP32-based real-time monitoring system designed for environmental applications. The study demonstrates how various environmental parameters such as temperature, humidity, and air quality can be collected using sensors and transmitted through the ESP32 module to cloud platforms for real-time monitoring and analysis. It highlights the system's reliability, low power consumption, and cost-effectiveness, making it suitable for large-scale deployment. This research is relevant to our project as it validates the use of ESP32 for building efficient, real-time environmental and industrial monitoring systems [5].

The research conducted by S. B. Younis et al. (2025) [6] focuses on the design of a low-cost IoT monitoring system integrated with cloud computing and augmented reality (AR). The study demonstrates how sensor data can be collected using IoT devices and transmitted to cloud platforms for real-time monitoring and analysis, while AR technology is used to visualize the

data in an interactive and user-friendly manner. It highlights the benefits of combining IoT with AR for improved data interpretation, remote monitoring, and decision-making. This research is relevant to our project as it supports the development of cost-effective, cloud-based monitoring systems with enhanced visualization capabilities [6].

The research conducted by A. Das et al. (2025) [7] presents an IoT-based smart monitoring system that integrates ESP32 with AI-based decision support. The study demonstrates how sensor data can be collected and transmitted in real time using ESP32, while AI algorithms analyze the data to provide intelligent insights and automated decision-making. It highlights the system's ability to detect anomalies, improve efficiency, and reduce human intervention through smart analytics. This research is relevant to our project as it supports the use of ESP32 combined with AI techniques for developing an intelligent, real-time industrial monitoring system [7].

The research conducted by Vinit et al. (2025) [8] presents an IoT-based monitoring system using ESP32 for real-time applications. The study demonstrates how sensor data can be collected and transmitted through the ESP32 module to enable continuous monitoring and instant data access. It highlights the system's efficiency, low cost, and reliability in handling real-time data for various applications. The research also emphasizes the importance of wireless communication and cloud connectivity for improving system performance. This research is relevant to our project as it supports the use of ESP32 for developing efficient, real-time industrial monitoring systems [8].

The research conducted by M. Abdullah et al. (2025) [9] presents a cellular IoT-based environmental monitoring system using ESP32 for real-time data transmission. The study demonstrates how environmental parameters such as temperature, humidity, and air quality can be monitored using sensors and transmitted over cellular networks, ensuring reliable communication even in remote areas without Wi-Fi access. It highlights the advantages of using cellular IoT for wider coverage, improved connectivity, and continuous monitoring. This research is relevant to our project as it supports the use of ESP32 with advanced communication technologies for building a robust and real-time industrial monitoring system [9].

The research conducted by D. Vaithiyanathan et al. (2024) [10] presents an ESP32-based industrial monitoring and alert system with cloud integration for real-time data tracking. The study demonstrates how industrial parameters such as temperature, humidity, and equipment status can be continuously monitored using sensors and transmitted to cloud platforms for storage and visualization. It also highlights the implementation of an alert mechanism that notifies users when abnormal conditions are detected, improving safety and response time. This research is relevant to our project as it validates the use of ESP32 and cloud integration for developing efficient industrial monitoring and alert systems [10].

The research conducted by J. Lee, B. Bagheri, and H. Kao (2022) [11] presents a cyber-physical systems (CPS) architecture for Industry 4.0 manufacturing. The study explains how physical machines, sensors, and digital systems are integrated through IoT and cloud technologies to enable real-time data exchange, monitoring, and intelligent decision-making. It highlights key concepts such as smart factories, predictive maintenance, and automation driven by data analytics. This research is relevant to our project as it provides a foundational framework for integrating IoT-based monitoring systems with cloud platforms to achieve efficient and intelligent industrial operations [11].

The research conducted by P. Singh and K. Mehta (2023) [12] presents a web-based dashboard for IoT monitoring developed using HTML, CSS, and JavaScript. The study demonstrates how data collected from IoT devices can be visualized in real time through interactive charts, graphs, and user-friendly interfaces on a web platform. It highlights the importance of responsive design, efficient data handling, and seamless integration with cloud databases for effective monitoring. This research is relevant to our project as it supports the development of a web dashboard for real-time visualization and control of industrial data using front-end technologies [12].

The documentation provided by Espressif Systems (2024) [13] presents the ESP32 Technical Reference Manual for the ESP32 DevKit V1. It explains the architecture, features, and functionalities of the ESP32 microcontroller, including its dual-core processor, Wi-Fi and Bluetooth capabilities, GPIO interfaces, and power management features. The manual also details communication protocols, memory organization, and peripheral configurations used in embedded system development. This reference is relevant to our project as it provides essential

technical knowledge required for implementing ESP32-based real-time monitoring and control systems [13].

The documentation provided by Google (2024) [14] explains the Firebase Realtime Database, a cloud-hosted NoSQL database that enables real-time data synchronization between clients and servers. It demonstrates how data can be stored, retrieved, and updated instantly across web and mobile applications using Firebase APIs. The documentation highlights features such as real-time updates, offline data support, scalability, and secure access through authentication and rules. This reference is relevant to our project as it supports the use of Firebase for real-time data storage, synchronization, and visualization in IoT-based monitoring systems [14].

The research conducted by D. Witczak (2024) [15] provides a comprehensive review of monitoring and control systems based on IoT. The study analyzes various IoT architectures, communication protocols, and cloud-based solutions used for real-time data monitoring and automation in industrial environments. It highlights the advantages of IoT systems such as improved efficiency, remote accessibility, scalability, and predictive maintenance capabilities. The paper also discusses challenges like data security, interoperability, and system reliability. This research is relevant to our project as it offers a broad understanding of IoT-based monitoring frameworks and supports the design of efficient and scalable industrial monitoring systems [15].

Chapter 3

Overview of System and Software Development

This section lays the groundwork for understanding the project. It begins with a general overview, providing a high-level understanding of the system's purpose and objectives. Following this, a block diagram visually represents the system's main components and their interconnections. Finally, the working of the project will describe the operational flow and how the system functions.

3.1 Overview of System:

The proposed system is a Smart Factory IoT Dashboard with Equipment Control designed to provide an efficient and user-friendly solution for real-time industrial monitoring and automation. The system is built around the ESP32 microcontroller, which acts as the central unit responsible for collecting, processing, and transmitting data. Sensors such as the DHT11 are used to measure environmental parameters like temperature and humidity, while an IR sensor is used for object detection or monitoring movement within the system. The ESP32 continuously reads this sensor data and sends it to the Firebase Realtime Database over a Wi-Fi network, ensuring that the data is stored and updated in real time. A web-based dashboard is developed using HTML, CSS, and JavaScript to display this data in a clear and interactive format, allowing users to monitor factory conditions remotely from any location. In addition to monitoring, the system also enables remote control of industrial equipment such as fans, lights, a conveyor system (represented by a DC motor), and a buzzer using a 4-channel relay module. The user can send control commands through the dashboard, which are stored in Firebase and then fetched by the ESP32 to operate the respective devices accordingly. This two-way communication between hardware and software ensures real-time synchronization and efficient system performance. The overall design focuses on reducing manual intervention, improving operational efficiency, enhancing safety, and providing a cost-effective and scalable solution for smart industrial environments aligned with the concept of Industry 4.0.

3.2 Block Diagram of Smart Factory IoT Dashboard with Equipment Control :

The block diagram shown in Fig. 3.1 shows the complete Workflow of the Smart Factory

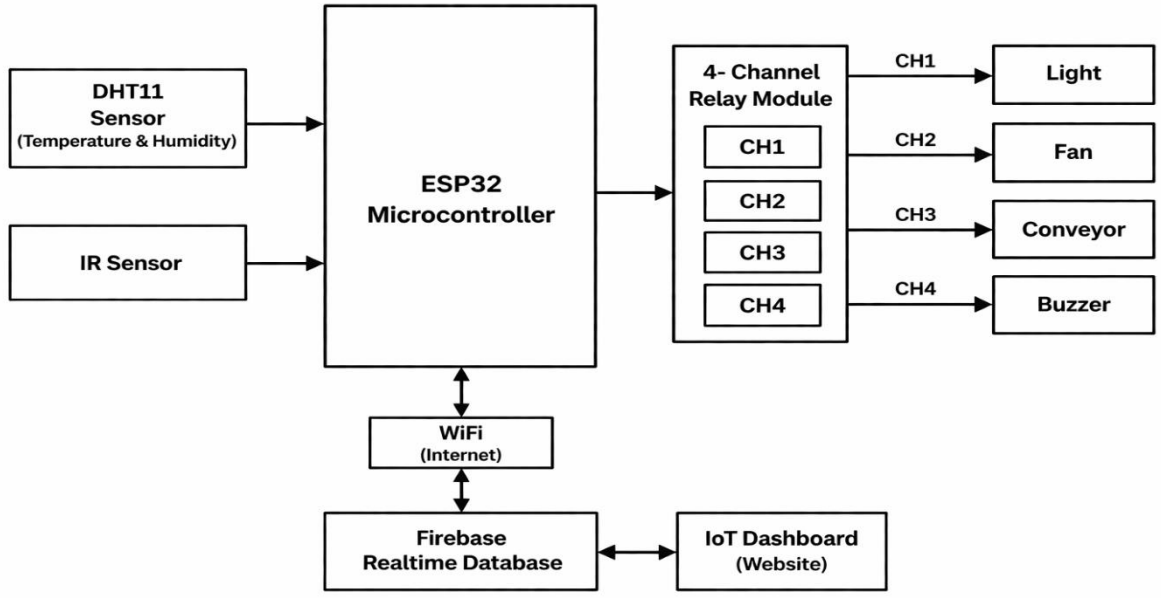


Fig. 3.1 Block Diagram of Smart Factory IoT Dashboard With Equipment Control

3.3 Description of the Block Diagram :

Power Supply Unit:

The power supply unit serves as the primary source of electrical energy for the entire Smart Factory IoT system. It typically consists of a DC power adapter or a regulated power supply circuit that provides a stable and consistent voltage to all components such as the ESP32 microcontroller, sensors, and relay module. Since different components operate at different voltage levels, proper regulation and distribution of power are essential to ensure smooth functioning. Any fluctuation in voltage can lead to system instability, unexpected resets, or even damage to sensitive components.

Therefore, this unit plays a crucial role in maintaining system reliability, safety, and long-term performance by ensuring that each component receives the appropriate power under all operating conditions.

Sensing Unit:

The sensing unit is responsible for collecting real-time environmental and operational data from the system, acting as the input layer of the entire setup. In this project, sensors such as the DHT11 and IR sensor are used to monitor different parameters. The DHT11 sensor measures temperature and humidity, providing continuous data about environmental conditions that may affect machine performance or safety. The IR sensor is used for object detection or motion sensing, which is useful for monitoring activity such as object presence on a conveyor or detecting obstacles. These sensors continuously gather data from the surroundings and send it to the ESP32 microcontroller for further processing. Thus, the sensing unit plays a vital role in enabling real-time monitoring and intelligent decision-making within the system.

Control Unit (ESP32 Microcontroller):

The ESP32 microcontroller functions as the central control and processing unit of the system, coordinating all operations between sensors, cloud, and output devices. It receives input data from the sensing unit, processes it based on programmed logic, and manages communication with the cloud platform. The ESP32 is programmed using Embedded C in the Arduino IDE, making it capable of handling multiple tasks such as data acquisition, wireless communication, and device control simultaneously. It continuously sends sensor data to the Firebase Realtime Database and also retrieves control commands from it. Based on these commands, the ESP32 activates or deactivates connected devices through the relay module.

Communication Unit (Wi-Fi Module):

The communication unit is responsible for enabling wireless data exchange between the ESP32 microcontroller and the cloud platform. In this system, the inbuilt Wi-Fi module of the ESP32 is used to establish a connection with a local wireless network. Through this connection, sensor data collected by the ESP32 is transmitted to the Firebase Realtime Database in real time, allowing users to monitor system parameters remotely. Similarly, control commands sent by the user through the web dashboard are received by the ESP32 via the same network.This bidirectional communication ensures that the system remains updated and responsive at all times. Reliable communication is essential for the proper functioning of IoT systems, and this unit ensures seamless data transfer and remote accessibility.

Cloud Unit (Firebase Realtime Database):

The Firebase Realtime Database acts as the central cloud platform for storing, managing, and synchronizing data within the system. It receives sensor data from the ESP32 and updates it instantly, making it available to the web dashboard in real time. At the same time, it stores control commands sent by the user, which are then accessed by the ESP32 for execution. One of the key advantages of Firebase is its ability to provide real-time synchronization, ensuring that any changes in data are immediately reflected across all connected devices. This makes the system highly efficient and responsive. The cloud unit essentially acts as a bridge between the hardware components and the user interface, enabling smooth communication and data flow throughout the system.

User Interface Unit (Web Dashboard):

The user interface unit consists of a web-based dashboard that allows users to monitor and control the system conveniently. It is developed using standard web technologies such as HTML, CSS, and JavaScript, making it accessible through any modern web browser. The dashboard displays real-time sensor data, including temperature and humidity readings, allowing users to observe environmental conditions continuously. It also shows the current status of connected devices, such as whether the fan, light, conveyor, or buzzer is ON or OFF. Users can interact with the system by sending control commands through the dashboard, which are then stored in Firebase and executed by the ESP32. This unit enhances usability, reduces manual effort, and enables efficient remote management of industrial equipment.

Relay Control Unit:

The relay control unit is an essential component that allows the low-power ESP32 microcontroller to control high-power electrical devices safely. Since the ESP32 operates at low voltage and cannot directly handle high-voltage loads, a relay module is used as an interface. In this project, a 4-channel

relay module is used, enabling independent control of multiple devices. Each relay acts as an electrically operated switch that opens or closes a circuit when triggered by the ESP32.

The microcontroller sends control signals to the relay based on commands received from the Firebase database. Additionally, the relay provides electrical isolation between the control circuit and the load circuit, ensuring safety and preventing damage to the microcontroller. This unit plays a critical role in converting digital signals into real-world actions.

3.4 Working of Model :

The Smart Factory IoT Dashboard system operates through the coordinated interaction of multiple units, including sensing, processing, communication, cloud storage, and control mechanisms. The ESP32 microcontroller acts as the core of the system, enabling real-time monitoring and remote control of industrial equipment. The system uses Wi-Fi communication and Firebase Realtime Database to ensure seamless data exchange between hardware and the user interface.

1. Powering the System:

The entire system is powered by a regulated power supply unit that provides stable DC voltage to all components, including the ESP32, sensors, and relay module. This unit ensures proper distribution of electrical energy so that each component operates within its required voltage range. A stable power supply is essential for maintaining continuous system operation, preventing unexpected resets, and protecting components from voltage fluctuations. Proper power management improves system reliability and ensures long-term performance of the IoT setup.

2. Data Acquisition through Sensors:

The system begins its operation by collecting real-time data using the sensing unit. Sensors such as the DHT11 measure environmental parameters like temperature and humidity, which are important for monitoring industrial conditions. Additionally, an IR sensor is used for object detection or motion sensing, which can be useful in automation processes such as conveyor monitoring. These sensors continuously capture data from the surroundings and send it to the ESP32 microcontroller for further processing. This step forms the foundation of real-time monitoring.

3. Processing and Control using ESP32:

The ESP32 microcontroller acts as the central processing unit of the system. It receives raw data from the sensors, processes it, and prepares it for transmission. The ESP32 is programmed using Embedded C in the Arduino IDE and is capable of handling multiple tasks such as reading sensor inputs, managing Wi-Fi communication, and controlling output devices. It continuously updates sensor data to the cloud database and simultaneously checks for any new control commands. Based on these commands, it makes decisions and controls connected devices accordingly.

4. Wireless Communication via Wi-Fi:

The ESP32 uses its built-in Wi-Fi module to establish a connection with a local network, enabling wireless communication with the cloud platform. Through this connection, sensor data is transmitted to the Firebase Realtime Database in real time. Similarly, control commands sent by the user from the web dashboard are received by the ESP32 through the same communication channel. This two-way communication ensures that the system remains updated and responsive, allowing users to monitor and control devices from remote locations.

5. Cloud Data Management using Firebase:

The Firebase Realtime Database acts as the central cloud platform that manages all system data. Sensor readings sent by the ESP32 are stored in the database and updated instantly, making them available to the web dashboard. At the same time, control commands issued by the user are also stored in Firebase. The ESP32 continuously monitors the database for changes and retrieves the latest commands. Firebase provides real-time synchronization, ensuring that data is updated instantly across all connected components. This makes the system efficient and suitable for industrial applications.

6. User Interaction through Web Dashboard:

The The web-based dashboard acts as a centralized control and monitoring platform, providing users with a seamless and interactive interface to manage the entire system efficiently. It is built using core web technologies such as HTML, CSS, and JavaScript, ensuring compatibility across different devices and browsers without requiring additional software installation. The dashboard continuously retrieves real-time data from Firebase, displaying important environmental parameters like temperature and humidity in a clear and user-friendly format. It also visually represents the operational status of connected devices, helping users quickly understand system conditions. Users can interact with the system through control buttons, which send commands (e.g., turning ON/OFF devices) that are instantly updated in the Firebase database. These commands are then fetched and executed by the ESP32, ensuring quick response and synchronization between user input and hardware action. This approach enables efficient remote monitoring and automation, significantly minimizing the need for manual supervision and improving overall system productivity.

7. Equipment Control using Relay Module:

The relay module is responsible for controlling high-voltage electrical devices using low-voltage signals from the ESP32. In this system, a 4-channel relay module is used, allowing multiple devices to be controlled independently. When the ESP32 receives a command from Firebase, it sends a signal to the appropriate relay channel. The relay then switches the connected device ON or OFF. The relay also provides electrical isolation between the microcontroller and high-power devices, ensuring safe operation of the system.

8. Execution through Output Devices:

The final stage of the system involves the operation of output devices such as a fan, LED bulb, conveyor motor, and buzzer. These devices are connected to the relay module and perform actions based on the control signals received from the ESP32. For example, the fan can be used for cooling, the conveyor for material movement, the bulb for indication, and the buzzer for alerts. These devices demonstrate how the system can automate real industrial processes, making operations more efficient and controlled.

9. Real-Time Monitoring and Control Cycle:

The system operates continuously in a loop of monitoring and control. Sensors collect data and send it to the ESP32, which uploads it to Firebase. The dashboard displays this data in real time for user monitoring. At the same time, user commands are sent from the dashboard to Firebase, retrieved by the ESP32, and executed through the relay module. This continuous two-way communication ensures that the system remains active, responsive, and efficient. It enables smart automation and real-time decision-making, making the system suitable for modern Industry 4.0 applications.

Chapter 4

Hardware and Software Implementation

4.1 Hardware Components:

ESP 32 -Dev V1:

The ESP32 DevKit V1 is the main microcontroller used in the system and acts as the central control unit. It has built-in Wi-Fi and Bluetooth capabilities, making it highly suitable for IoT applications. The ESP32 reads data from sensors, processes it, and communicates with the Firebase Realtime Database over the internet. It also controls the relay module based on user commands. Due to its high processing power, low cost, and wireless connectivity, it is an ideal choice for real-time monitoring and automation systems.

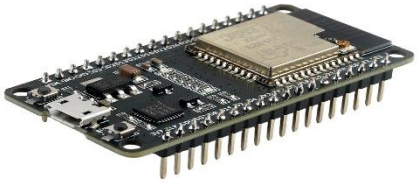


Fig.4.1 ESP 32 – Dev V1

DC Motor (60 rpm):

The 60 RPM DC motor is used to simulate or implement a conveyor system in the project. It is a low-speed, high-torque motor that rotates at approximately 60 revolutions per minute, making it suitable for controlled movement applications. The motor operates on low voltage and can be easily controlled using a relay module. It is commonly used in automation and robotics where steady and reliable motion is required.



Fig. 4.2 DC Motor (60 rpm)

DHT-11 (Temperature and Humidity) Sensor:

The DHT11 is a low-cost digital sensor used to measure temperature and humidity in the environment. It provides calibrated output directly in digital form, which makes it easy to interface with the ESP32 without requiring additional signal processing. In this project, it is used to monitor environmental conditions inside the factory setup. Although it has moderate accuracy, it is suitable for basic monitoring applications and helps in maintaining safe and efficient working conditions.

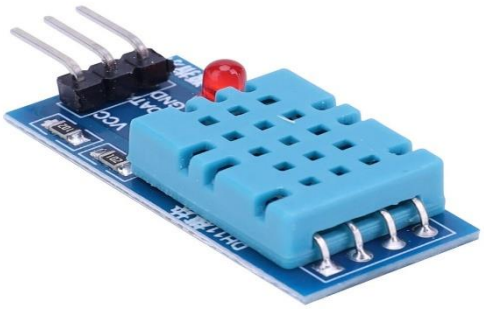


Fig. 4.3 DHT -11 Sensor

8. Buzzer:

The buzzer is an electronic sound-producing device used for alerts and notifications. It generates sound when a voltage signal is applied. In this project, it can be used to indicate specific conditions such as system alerts, object detection, or abnormal environmental changes. It is a simple yet effective component for providing audio feedback in automation systems.

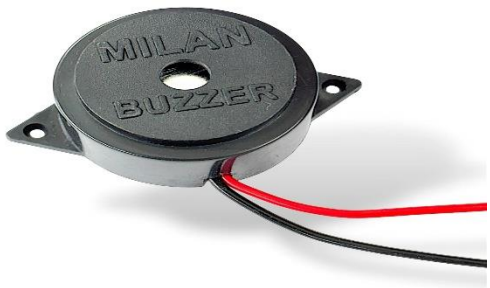


Fig. 4.4 Buzzer

IR Sensor:

The IR (Infrared) sensor is used for object detection and motion sensing in the system. It works by emitting infrared light and detecting its reflection from nearby objects. When an object is detected, the sensor outputs a digital signal (HIGH or LOW) to the ESP32. In this project, it can be used for applications like conveyor monitoring or detecting the presence of objects. Its simple working and easy interfacing make it a useful component in automation systems.

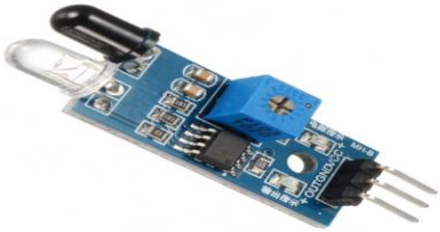


Fig. 4.5 IR sensor

4-Channel Relay module:

The 4-channel relay module is used to control multiple electrical devices using signals from the ESP32. Each relay acts as an electrically operated switch that can turn a device ON or OFF. Since the ESP32 operates at low voltage, the relay module helps in safely controlling high-voltage appliances like fans and motors. It also provides electrical isolation between the control circuit and load circuit, ensuring safety and protecting the microcontroller from damage.

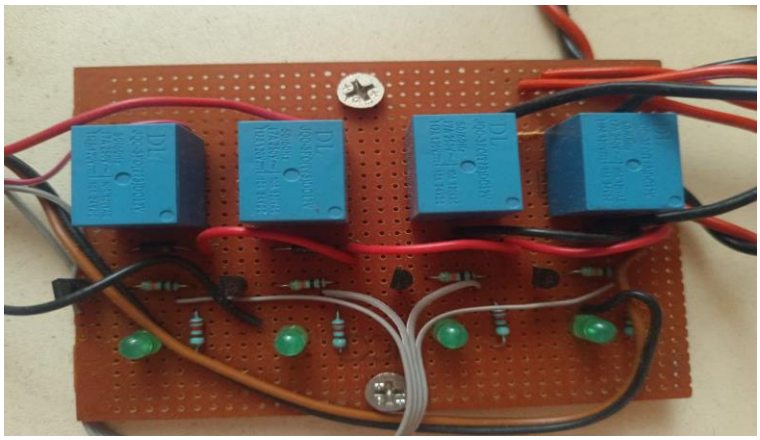


Fig. 4.6 4-Channel Relay Module

Single Relay :

Relay is acting as a controlled switch between the AC source and the load. The AC line is connected through the relay terminals, so the load (like a bulb or fan) will only receive power when the relay allows it. On the right side, the microcontroller is connected to the relay module using three pins: VCC (power), GND, and GPIO (control signal). When the microcontroller sends a signal through the GPIO pin, it activates the relay coil inside the module. This causes the internal switch to close, completing the AC circuit and turning ON the load. When the signal is removed, the relay opens the circuit, cutting off power and turning the load OFF. This setup allows a low-voltage microcontroller to safely control high-voltage AC devices without direct electrical contact.

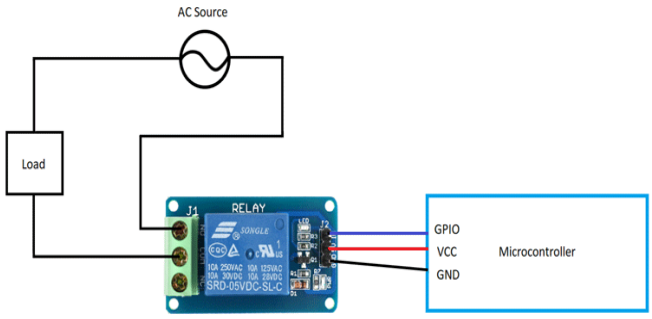


Fig 4.7 Single Relay

LED Bulb Zero Watt:

The zero watt LED bulb is used as an indicator or lighting device in the system. Despite its name, it consumes a very small amount of power and is suitable for continuous operation. In this project, it represents a lighting system that can be controlled remotely through the relay module. It helps demonstrate how electrical appliances can be automated using IoT technology.



Fig. 4.8 LED Bulb Zero Watt

Fan :

The fan is used as an output device to demonstrate control of electrical appliances. It can be switched ON or OFF using the relay module based on commands from the ESP32. In practical applications, such a fan can be used for cooling or ventilation in industrial environments. This component helps in showcasing real-world automation where environmental conditions can trigger device operation.



Fig. 4.9 Fan

Power Supply (Adapter) :



Fig. 4.10 DC Adapter

The power supply unit provides the necessary electrical energy required for the operation of all components in the system. It typically includes a DC adapter that converts AC mains supply into regulated DC voltage. Different components such as ESP32, sensors, and relay module require stable voltage for proper functioning. A reliable power supply ensures smooth operation, prevents system failure, and protects components from damage due to voltage fluctuations and provides the power.

4.2 Simulation:

Wokwi was used as the primary simulation tool for demonstrating and validating the working of the Smart Factory IoT Dashboard system. It is an online simulation platform that supports microcontrollers like ESP32 and allows testing of embedded systems in a virtual environment. Using Wokwi, we simulated the interfacing of components such as sensors and relay modules with the ESP32 and verified the logical operation of the system. The platform enabled us to run the embedded code, observe serial output, and test device control logic without requiring physical hardware. This helped in identifying errors, debugging the system, and ensuring that the core functionality of the project was working correctly before hardware implementation.

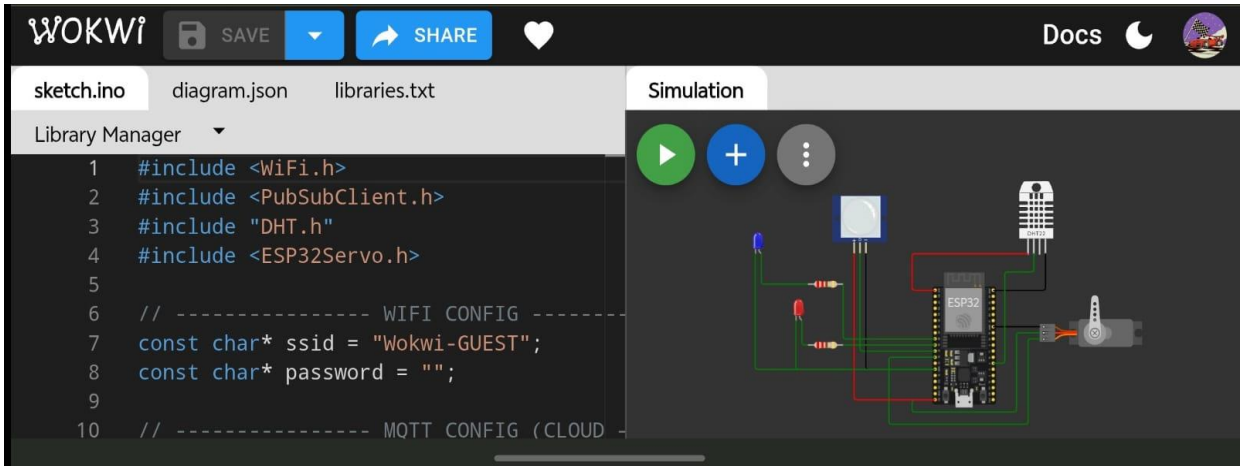


Fig. 4.11 Wokwi Simulation

Advantages of Using Wokwi for Simulation:

The use of Wokwi for simulation provided several advantages in the development of the system. Firstly, it offered a controlled environment where the system could be tested safely without the risk of hardware damage or incorrect wiring. This made it easier to experiment with different configurations. Secondly, it simplified the process of testing and debugging, as errors in the code could be quickly identified using the serial monitor and corrected accordingly. Additionally, the simulation allowed repeatability of tests, meaning the same conditions could be recreated multiple times to verify consistent system behavior. It also improved time and cost efficiency by reducing the need for repeated hardware setup during the initial stages of development. Overall, Wokwi helped in validating the core logic of sensor data processing and device control before moving to real hardware.

Simulation Working and Observations:

The simulation was carried out by designing a virtual circuit in Wokwi where the ESP32 microcontroller was connected with simulated input and output components. The ESP32 was programmed using Embedded C to read input signals and control outputs based on predefined conditions. During the simulation, virtual inputs were used to represent sensor behavior, and the ESP32 processed this data accordingly. Based on the programmed logic, the ESP32 generated output signals that simulated the operation of relay-controlled devices such as a fan, light, or buzzer. The serial monitor in Wokwi was used to observe real-time outputs, confirming that the ESP32 was correctly reading inputs and executing commands. The ON and OFF switching behavior of devices was successfully demonstrated, verifying that the system logic was functioning as intended.

Limitations of Simulation:

Despite its usefulness, the simulation has certain limitations. Wokwi does not fully support real-time cloud communication with platforms such as Firebase Realtime Database, which is a key part of the actual system. As a result, complete end-to-end IoT functionality cannot be replicated in the simulation environment. Additionally, real-world factors such as Wi-Fi connectivity issues, sensor inaccuracies, electrical noise, and power supply variations cannot be accurately modeled. The behavior of physical components like relays and motors may also differ from their simulated counterparts. Therefore, while simulation is useful for initial validation, final testing and verification must be carried out using actual hardware.

Conclusion of Simulation:

The simulation performed using Wokwi successfully demonstrated the basic functionality and logical operation of the Smart Factory IoT Dashboard system. It confirmed that the ESP32 was able to process inputs and generate appropriate outputs based on the programmed conditions. Although the simulation had limitations in terms of cloud integration and real-world behavior, it played an important role in validating the system design and ensuring that it was ready for hardware implementation. Thus, simulation served as an effective preliminary step in the development of the project.

4.3 Programming Language and Softwares Used:

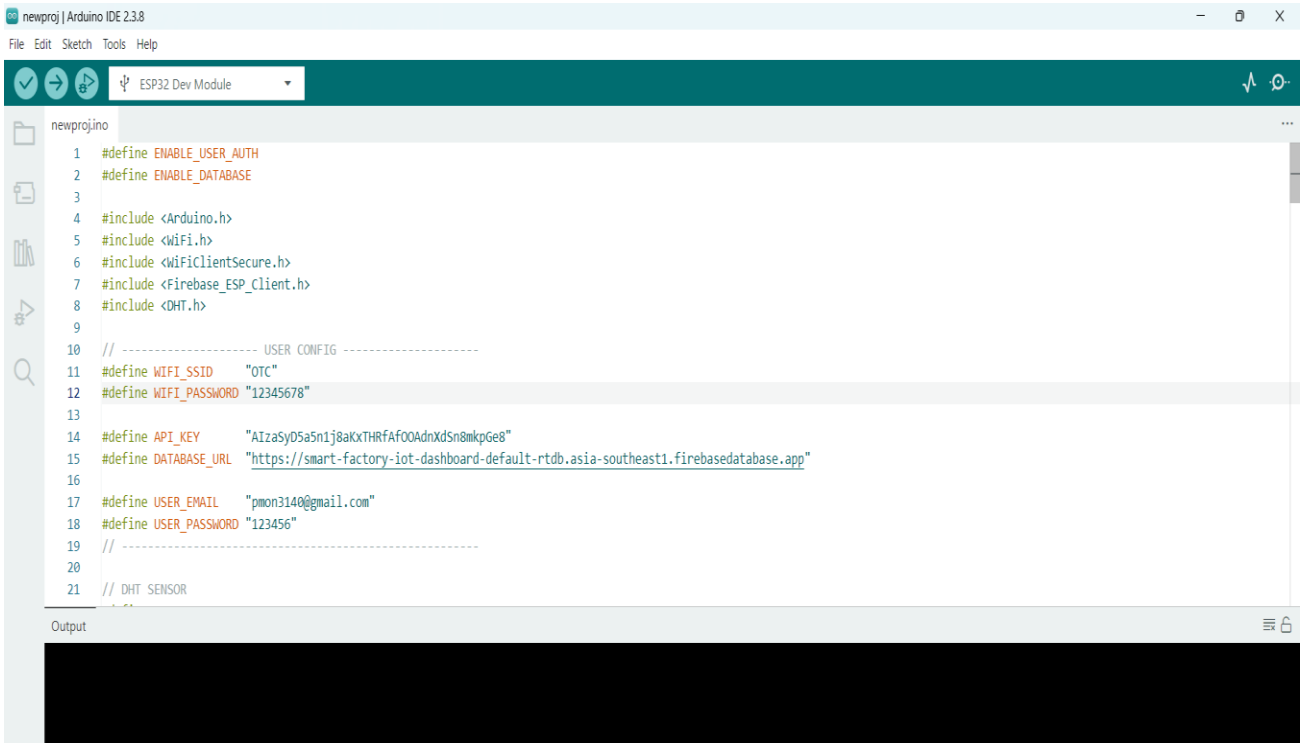
The Smart Factory IoT Dashboard system is developed using an integrated combination of embedded programming, cloud-based data management, and web technologies to enable real-time monitoring and control of industrial equipment. The system relies on the ESP32 microcontroller, which is programmed using Embedded C through the Arduino IDE to interface with sensors, process data, and communicate with external platforms. Sensor data such as temperature, humidity, and machine status is continuously collected and transmitted to the Firebase Realtime Database over a Wi-Fi network, where it is stored and synchronized in real time. A web-based dashboard, developed using HTML, CSS, and JavaScript, retrieves this data from Firebase and presents it to the user in an organized and interactive manner, allowing remote monitoring of system parameters. At the same time, the dashboard enables users to send control commands, which are updated in the database and fetched by the ESP32 to operate devices through relay modules. This bidirectional communication ensures seamless interaction between hardware and software components, forming a complete IoT ecosystem. Additionally, tools such as the Arduino IDE, web browsers, and Wokwi simulator are used for development, testing, and validation of the system, ensuring reliability and efficiency before deployment. Overall, the integration of these programming languages and software tools provides a scalable, cost-effective, and user-friendly solution aligned with modern Industry 4.0 applications.

4.3.1 Embedded Programming using Arduino IDE

Arduino IDE is used as the primary development platform for writing, compiling, and uploading the Embedded C (.ino) code to the ESP32 microcontroller. It provides a simple interface along with extensive support for libraries required in IoT applications, making it highly suitable for this project. In the initial stage, the ESP32 is configured within the Arduino IDE by installing the necessary board packages and selecting the correct communication port. The program includes the inclusion of required libraries for Wi-Fi connectivity, sensor interfacing, and Firebase communication. The setup function of the code is used to initialize serial communication for debugging, configure input-output pins, and establish a connection to the Wi-Fi network. Once connected, the ESP32 is linked to the Firebase Realtime Database using appropriate authentication credentials, enabling cloud

communication.

The main functionality of the system is implemented in the loop section of the program, where continuous operations are performed. The ESP32 reads real-time data from sensors such as the DHT11 for temperature and humidity, and the IR sensor for object detection. This data is processed and uploaded to the Firebase database at regular intervals, ensuring that the web dashboard receives updated information. Simultaneously, the ESP32 monitors the database for any changes in control values sent by the user. When a command is detected, such as turning a device ON or OFF, the ESP32 processes the instruction and sends the appropriate signal to the relay module. This activates or deactivates connected devices like fans, lights, conveyors, or buzzers. The Arduino IDE also provides debugging support through the serial monitor, where sensor readings, connection status, and control actions can be observed in real time. This helps in troubleshooting errors and optimizing system performance. Overall, the embedded programming implemented using Arduino IDE forms the core logic of the system, enabling seamless interaction between sensors, cloud database, and output devices.



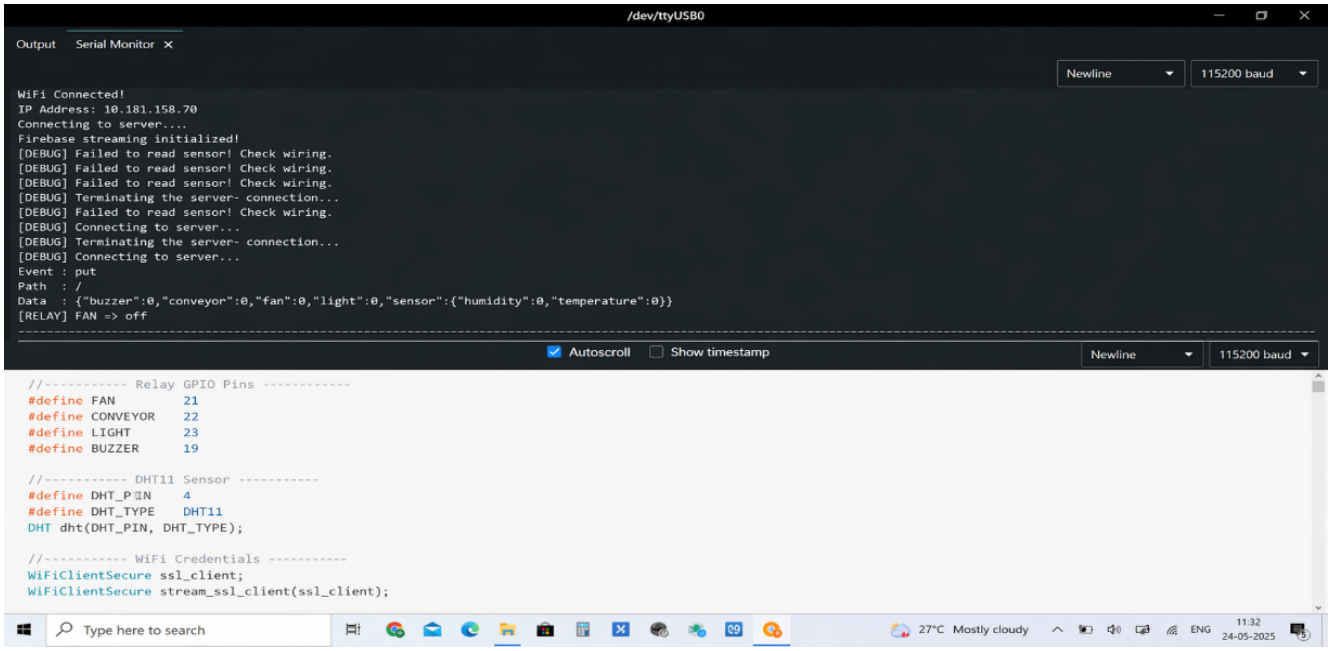
4.12 Embedded Programming using Arduino IDE

4.4 Arduino IDE Code Upload

Arduino IDE Code Upload

The Arduino IDE is used to upload the developed Embedded C (.ino) program to the ESP32 microcontroller, enabling it to execute all IoT-based operations defined in the project. Before uploading, the ESP32 board is configured in the Arduino IDE by installing the required board package, selecting the correct board type (ESP32 Dev Module), and choosing the appropriate COM port. The program includes necessary libraries for Wi-Fi connectivity, sensor interfacing, and Firebase communication, which must be properly included and verified. The code is first compiled using the "Verify" option in the Arduino IDE to check for syntax errors, missing dependencies, or logical issues. This step ensures that the program is error-free and ready for execution. Once verified, the ESP32 is connected to the system using a USB cable, and the upload process is initiated using the "Upload" option. During this process, the compiled code is transferred to the flash memory of the ESP32 through a serial communication interface.

After the successful upload of the code, the ESP32 starts executing the program automatically without requiring any external trigger. The Arduino IDE provides a serial monitor that is used to observe real-time system behavior, including sensor readings, Wi-Fi connection status, IP address allocation, and communication with the Firebase databaseallowing for continuous improvement and testing.

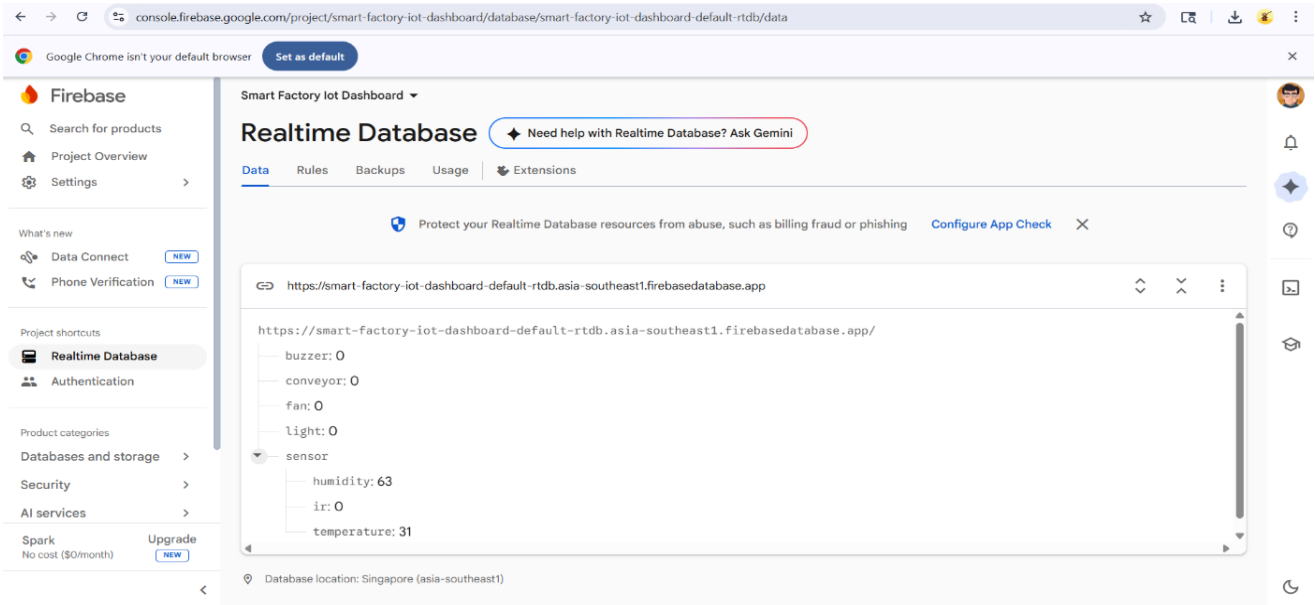


4.13 Arduino Ide Code Upload

4.5 Cloud Platform – Firebase Realtime Database

Firebase Realtime Database is used as the central cloud platform for storing, managing, and synchronizing all system data in the Smart Factory IoT Dashboard. It is a cloud-hosted NoSQL database that stores data in a JSON format and provides real-time synchronization between connected devices. In this project, the ESP32 microcontroller continuously sends sensor data such as temperature, humidity, and IR-based object detection status to the Firebase database over a Wi-Fi connection. Each parameter is stored under specific nodes in a structured format, making it easy to organize and access data. This allows the web dashboard to fetch and display live data, enabling users to monitor industrial conditions continuously.

In addition to data storage, Firebase also plays a crucial role in device control and system interaction. The web dashboard updates control values such as ON/OFF status of devices (fan, light, conveyor, buzzer) in the database when a user performs an action. When a change is detected, the ESP32 processes the command and activates the corresponding relay to control the device. Firebase also provides features such as secure authentication, data access rules, and reliable cloud hosting, ensuring safe and efficient operation of the system. Its scalability allows the system to handle multiple devices and users without performance issues. Overall, Firebase Realtime Database acts as a bridge between the hardware and the user interface, enabling seamless two-way communication and making the system truly IoT-enabled.



4.14 Cloud Platform- Firebase Realtime Database

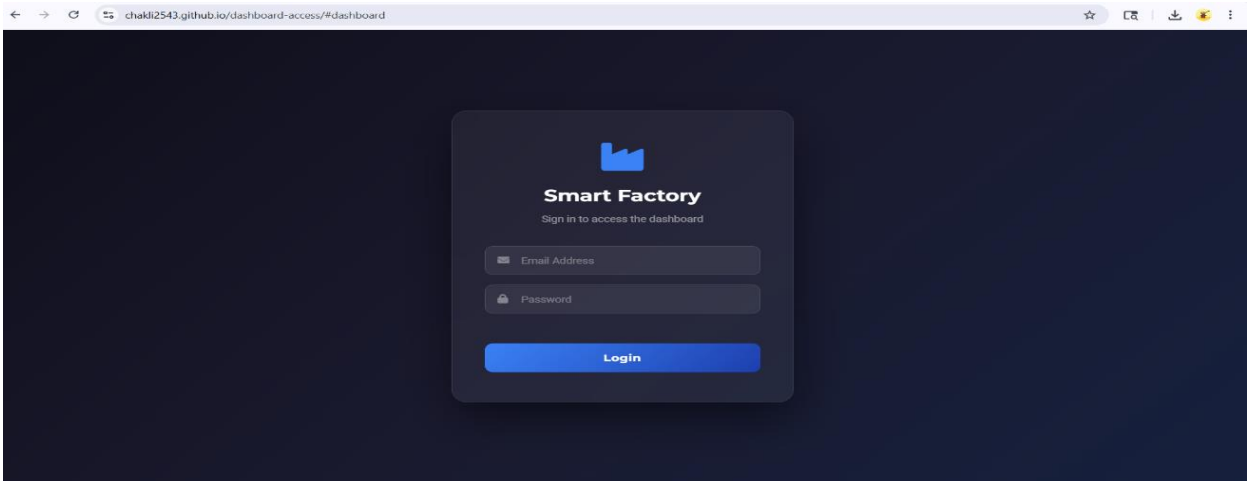
4. 6 Web Dashboard Interface

The web dashboard serves as the main user interface of the Smart Factory IoT Dashboard system and is developed using HTML, CSS, and JavaScript. HTML is used to create the basic structure of the webpage, including sections for displaying sensor values, device status, and control buttons. It defines elements such as headings, tables, cards, and buttons that organize the information in a clear and readable format. This structured layout allows users to easily view real-time data such as temperature, humidity, and machine status. The design ensures that all important parameters are visible at a glance, making monitoring simple and effective.

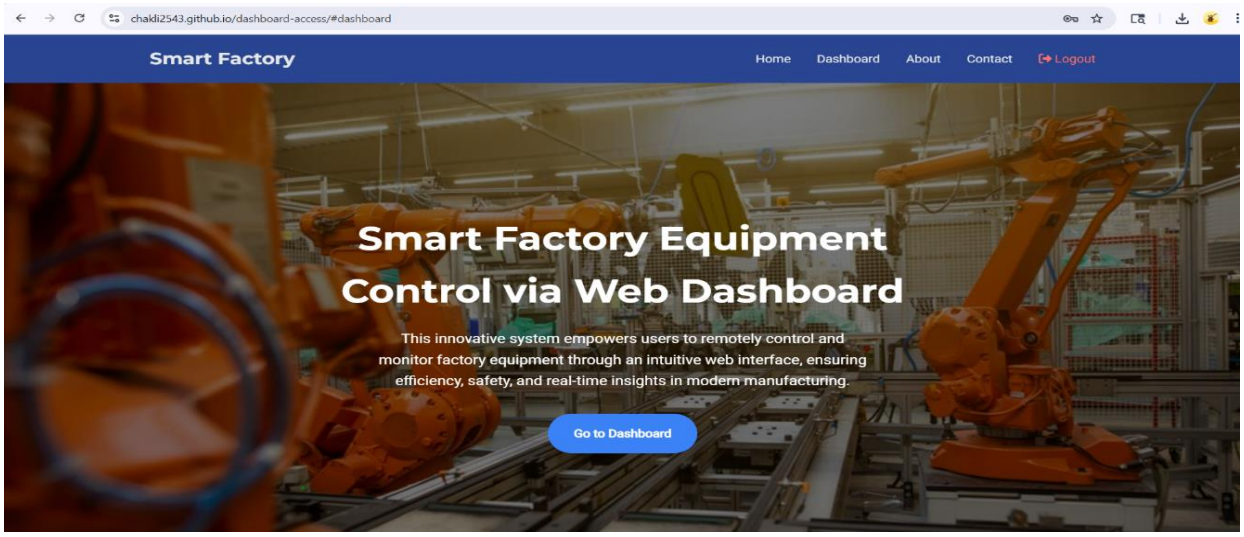
CSS is used to enhance the visual appearance and layout of the dashboard, making it more user-friendly and professional. It controls aspects such as colors, fonts, spacing, alignment, and responsiveness of the webpage. With proper styling, the dashboard becomes visually appealing and easy to navigate, even for users with minimal technical knowledge. CSS also helps in creating responsive designs so that the dashboard can be accessed on different devices such as laptops, tablets, and mobile phones without affecting usability. This improves accessibility and ensures that users can monitor and control the system from anywhere.

JavaScript is used to add dynamic functionality and enable real-time interaction between the dashboard and the Firebase Realtime Database. It is responsible for fetching live sensor data from Firebase and updating the display automatically without refreshing the page. It also handles user actions such as button clicks for turning devices ON or OFF. When a user interacts with the dashboard, JavaScript sends the corresponding command to Firebase, which is then processed by the ESP32. This real-time communication ensures smooth and instant control of devices. Overall, the combination of HTML, CSS, and JavaScript creates an interactive, responsive, and efficient web dashboard for monitoring and controlling the IoT system.

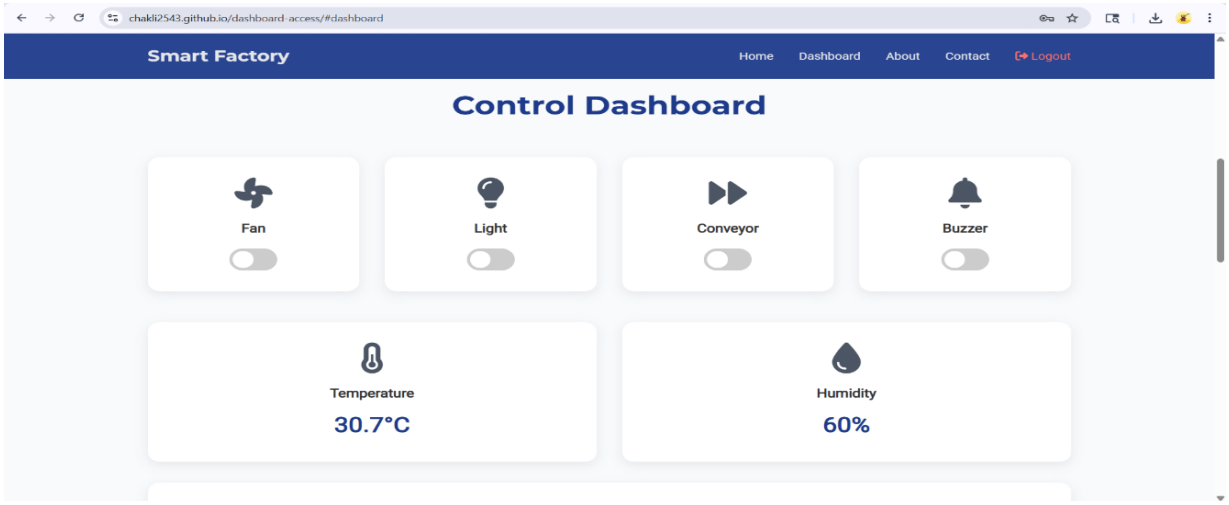
Additionally, the web dashboard is designed with simplicity and efficiency in mind, ensuring that even non-technical users can operate the system. Clear labels, intuitive controls, and real-time feedback make it easy to understand system status and take quick actions when needed. The dashboard can also be further enhanced by adding features such as alerts, data logs, or graphical representations of sensor values, which would improve analysis and decision-making. This flexibility makes the web interface scalable and adaptable for more advanced industrial applications.



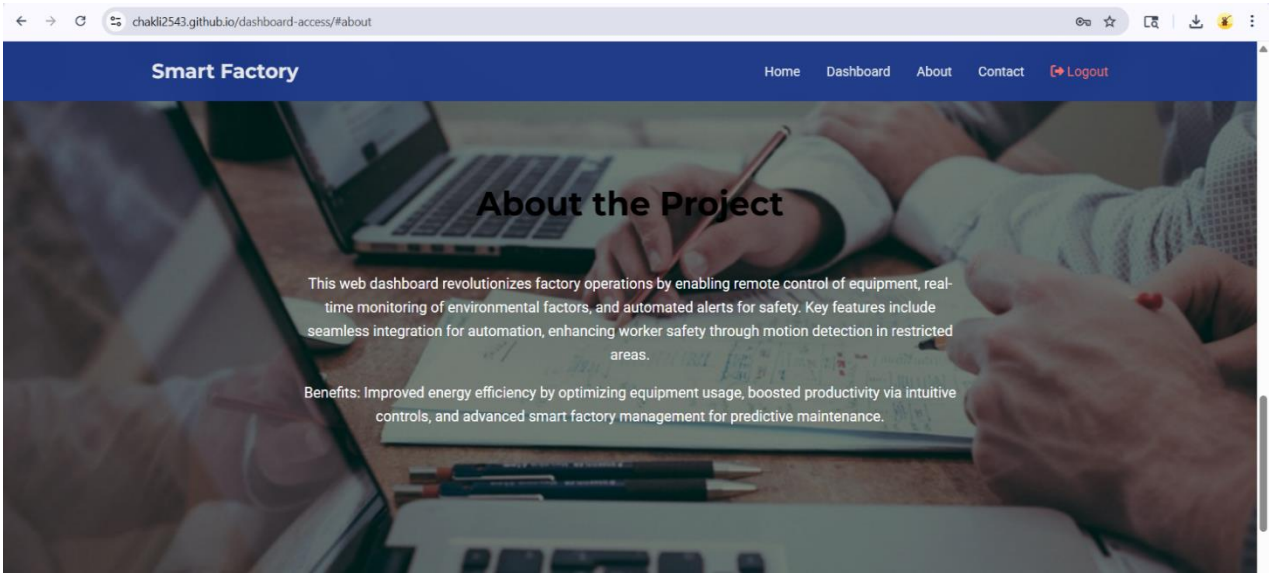
4.15 Login page



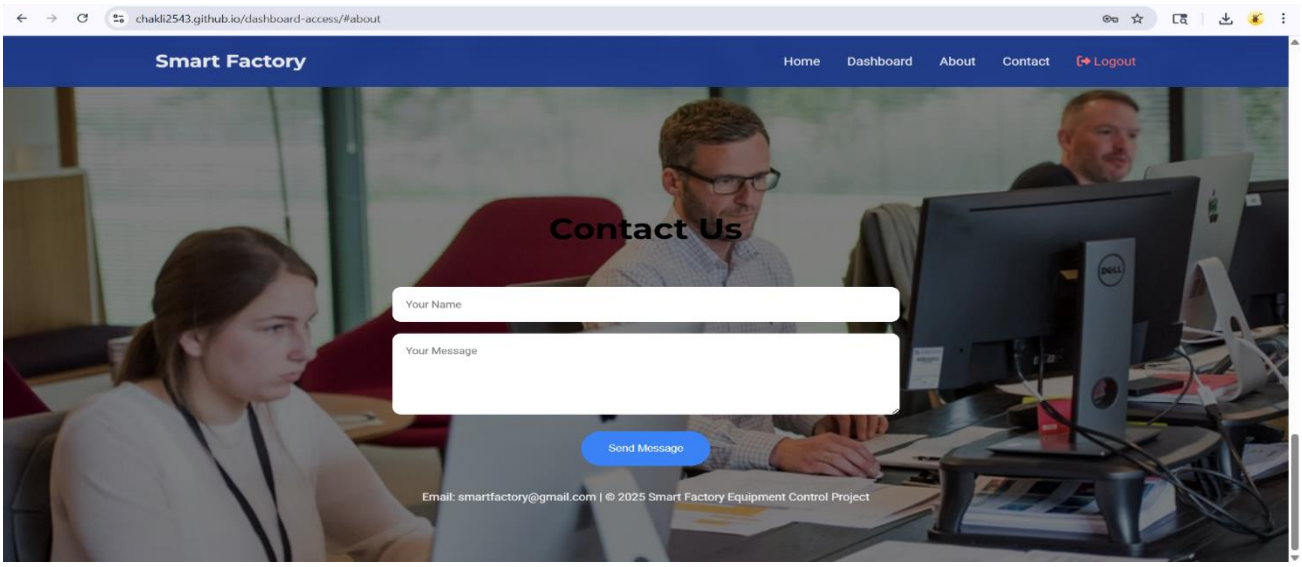
4.16 Home Page



4.17 Control Dashboard



4.18 About the Project

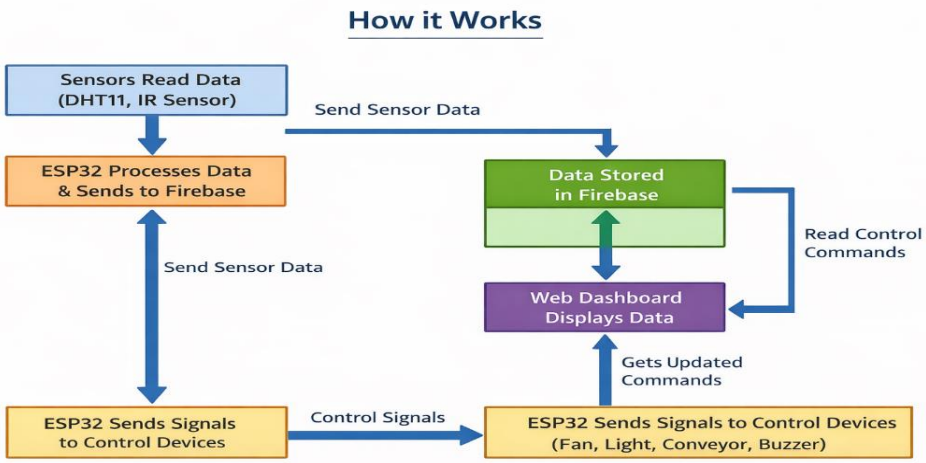


4.19 Contact us Page

4.7 System Dataflow :

The Smart Factory IoT Dashboard system follows a continuous data flow that enables real-time monitoring of industrial parameters. The process begins with sensors such as the DHT11 and IR sensor, which collect environmental and operational data like temperature, humidity, and object detection. This data is read and processed by the ESP32 microcontroller, which then transmits it to the Firebase Realtime Database using its built-in Wi-Fi module. The database stores the data in a structured format and updates it instantly. The web dashboard retrieves this data from Firebase in real time and displays it to the user, allowing continuous monitoring of system conditions from any location.

In the control flow, when a user interacts with the web dashboard and sends a command (such as turning a device ON or OFF), the command is stored in the Firebase database. The ESP32 continuously checks for these updates and retrieves the latest control values. Based on the received command, the ESP32 sends signals to the relay module, which then switches the connected devices accordingly. This creates a two-way communication system where data flows from sensors to the user and control signals flow from the user back to the system. This continuous and synchronized data exchange ensures efficient, real-time operation of the entire system.



4.20 System Dataflow

4.8 Operating System:

The development of the Smart Factory IoT Dashboard system is carried out using operating systems such as Windows or Linux, which provide a stable and efficient environment for software development and testing. These operating systems support all the required tools, including Arduino IDE for embedded programming, web browsers for testing the dashboard, and simulation platforms like Wokwi. They also provide necessary drivers and communication support for connecting the ESP32 microcontroller via USB and enabling serial communication during code upload and debugging. The user-friendly interface and wide compatibility of these operating systems make them suitable for handling both hardware and software components of the project.

In addition to supporting development tools, the operating system plays a key role in managing network connectivity and system resources. It enables smooth Wi-Fi configuration, allowing the ESP32 to communicate with the Firebase Realtime Database over the internet. The operating system also ensures efficient execution of background processes such as running the web dashboard locally, testing APIs, and handling multiple development tasks simultaneously. Its reliability and flexibility contribute to a seamless development workflow, making it easier to design, test, and implement the complete IoT-based system.

4.9 Flow-Chart :

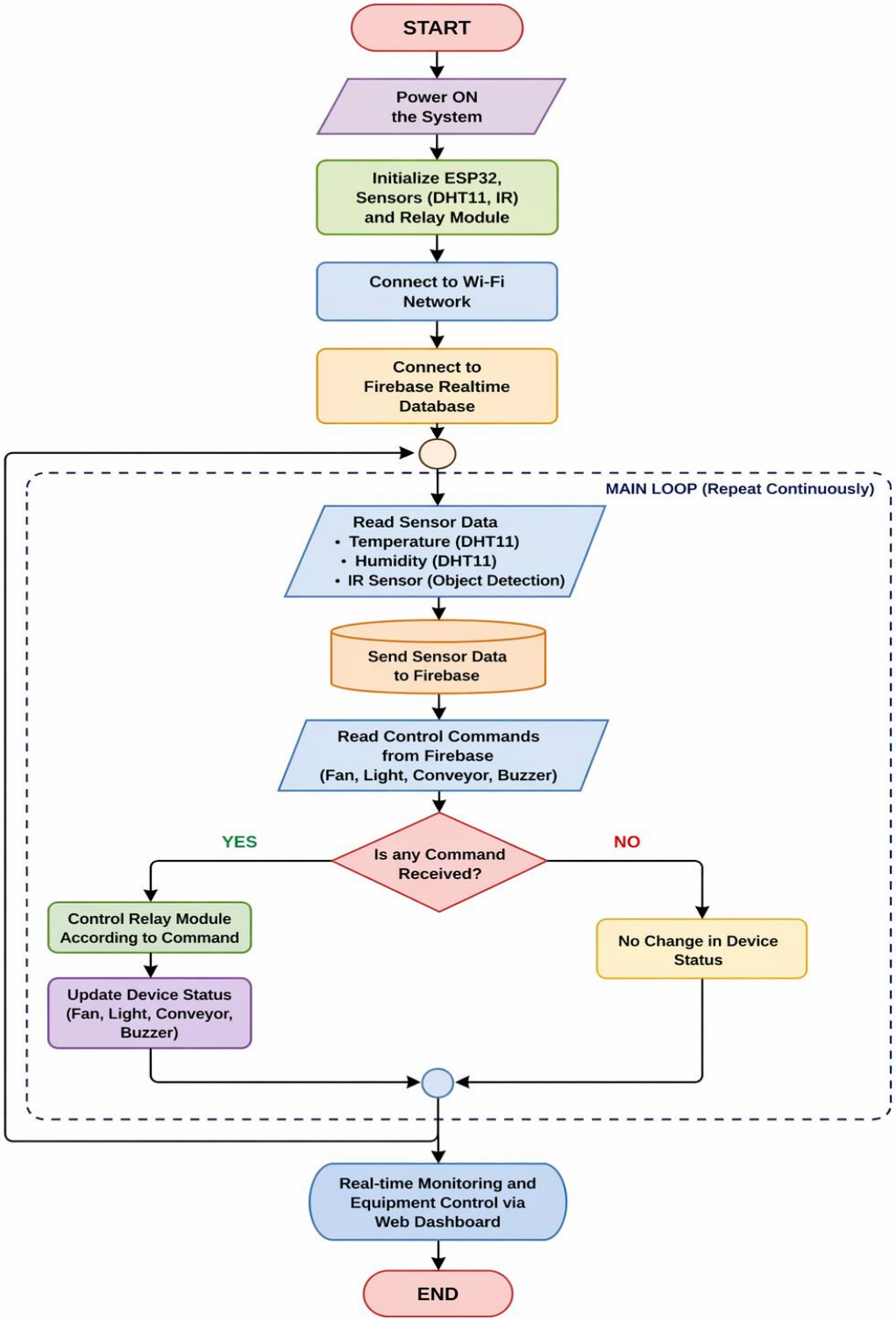


Fig 4.21 Flowchart

Chapter 5

Results and Discussion

5.1 Hardware Implementation Output Images:

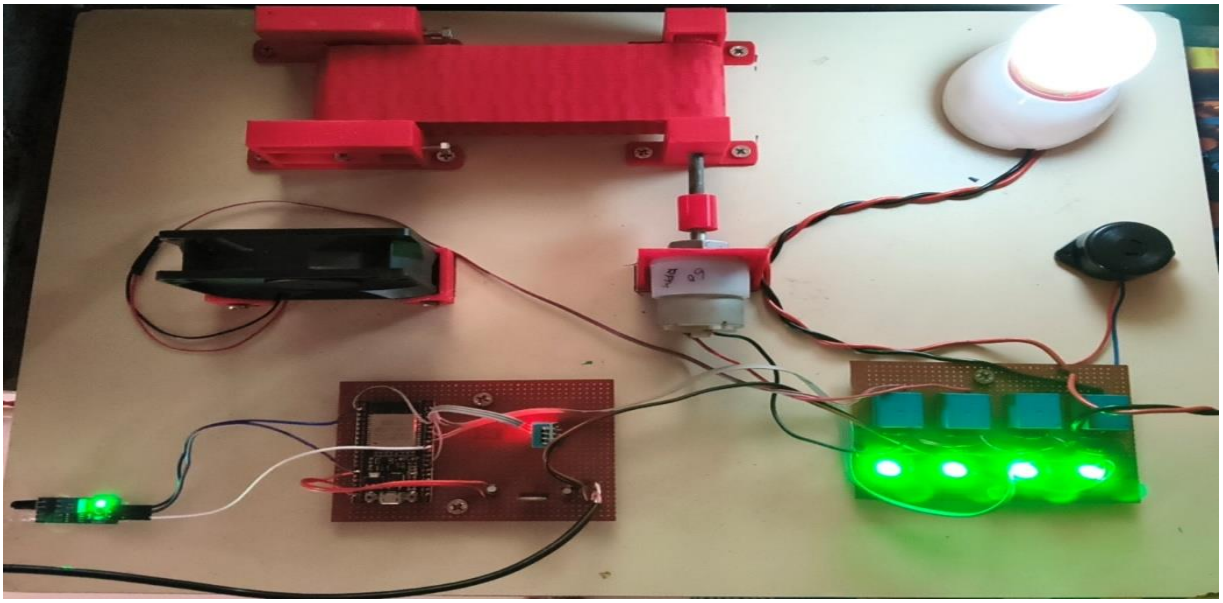


Fig. 5.1 Smart Factory Iot Dashboard With Equipment Control

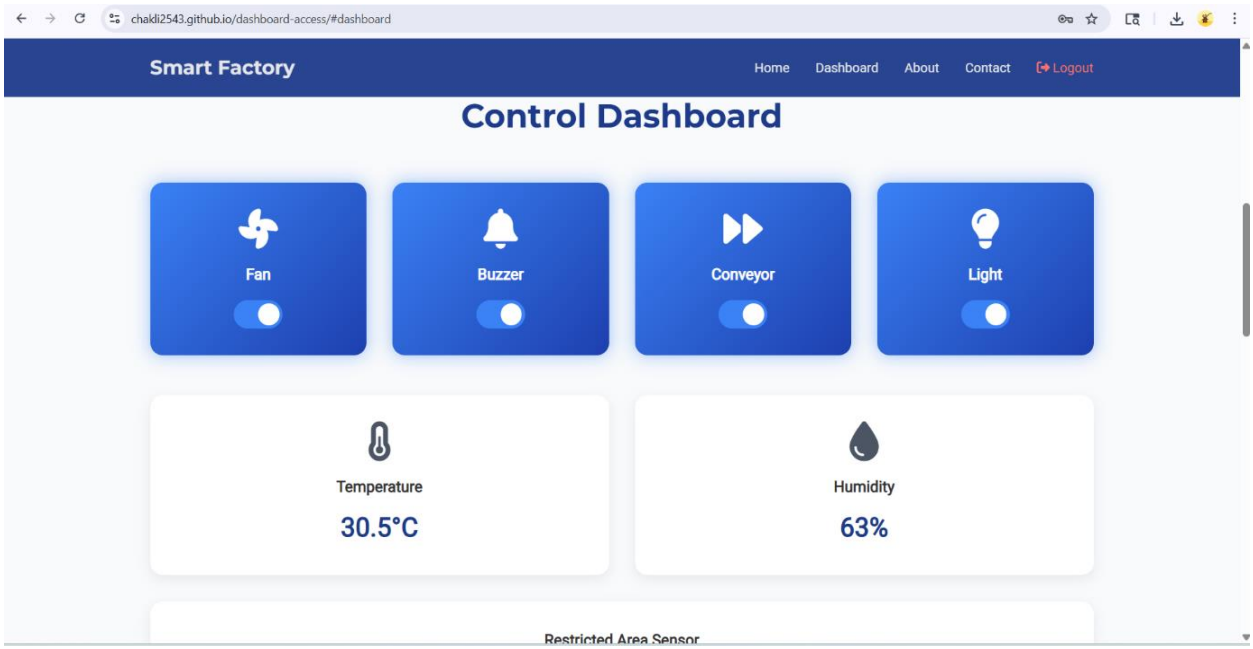


Fig. 5.2 Web Dashboard

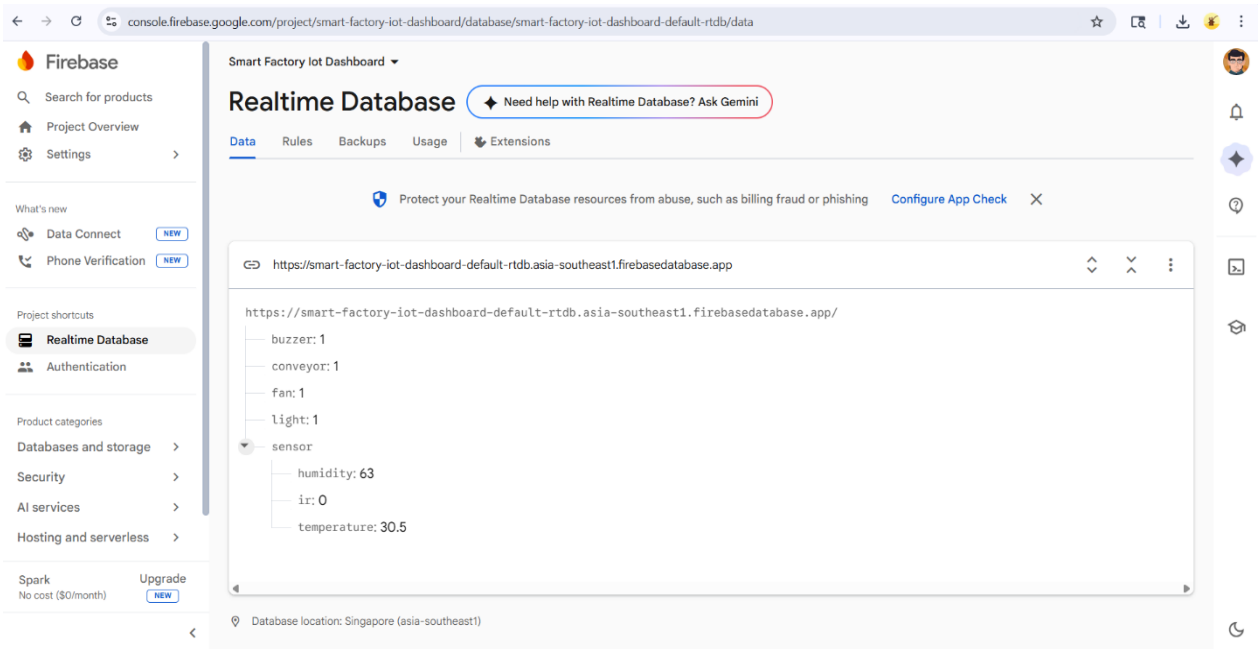


Fig. 5.3 Firebase Realtime Database

Future Scope

Future Scope

1. **AI Integration for Smart Decision-Making**

   Future enhancement can include integrating AI and machine learning algorithms to analyze real-time and historical data collected from the system. This will help in predicting equipment failures, identifying abnormal behavior, and optimizing machine performance. Such intelligent features will reduce downtime and improve overall productivity in industrial environments.

2. **Improved Data Processing and Management**

   The system can be improved by implementing proper data filtering, validation, and structured storage techniques in Firebase. This will ensure that only accurate and meaningful data is stored and displayed on the dashboard. Better data handling will also reduce errors, improve system reliability, and make analysis more effective.

3. **Mobile Application Development**

   A dedicated mobile application can be developed to provide users with easy access to the system from anywhere. The app can display real-time data, send instant alerts, and allow remote control of equipment. This will enhance user convenience and ensure faster response to critical situations in the factory.

4. **Scalability for Large Industrial Use**

   The project can be expanded to support multiple machines and sensors by connecting several ESP32 modules to the Firebase database. This will allow centralized monitoring and control of large-scale industrial operations. It will make the system suitable for real-world smart factory implementations.

5. **Advanced Automation Using Sensors**

   Future improvements can include fully automated control of equipment based on sensor inputs such as temperature, humidity, or machine load. The system can automatically trigger actions when certain conditions are met, reducing the need for manual intervention. This will improve efficiency, accuracy, and safety in industrial processes.

Conclusion

The **Smart Factory IoT Dashboard with Equipment Control** project successfully demonstrates the effective integration of IoT and cloud technologies to develop a reliable, scalable, and cost-efficient industrial monitoring and control system. By utilizing the ESP32 microcontroller, sensors, and the Firebase Realtime Database, the system enables seamless real-time data transmission and remote equipment control with minimal delay. The web-based dashboard, developed using HTML, CSS, and JavaScript, provides an intuitive interface for monitoring machine status and controlling operations efficiently. The implementation ensures stable communication between hardware and software components, allowing users to manage industrial processes with improved accuracy and reduced manual effort. This project not only fulfills the current requirements of smart monitoring and control but also establishes a strong foundation for future enhancements such as advanced automation, improved data handling, and intelligent system optimization. Ultimately, it highlights the significant role of IoT in transforming traditional industries into smart, connected, and efficient manufacturing environments aligned with Industry 4.0.

References

[1] M. S. Bakare et al., "IoT-Based Indoor Environmental Monitoring System Using ESP32 and Cloud Integration," *Springer Journal of Smart Systems,* Vol. 3, 2026, pp. 1–15.

[2] S. Sharma et al., "Intelligent Low-Cost IoT Monitoring System Using Firebase Cloud Platform," *Nature Scientific Reports,* 2026.

[3] Abdul Hasib and A. S. M. A. S. Akib, "Cloud-Enabled IoT System for Real-Time Monitoring and Control Using Firebase," *arXiv,* 2026.

[4] Abdul Hasib et al., "HydroSense: IoT Framework for Real-Time Monitoring with Cloud Integration," *arXiv,* 2026.

[5] I. N. Afifah et al., "ESP32-Based Real-Time Monitoring System for Environmental Applications," *Journal of Smart Systems and Control,* 2026.

[6] S. B. Younis et al., "Design of Low-Cost IoT Monitoring System with Cloud and AR Integration," *EPSTEM Journal,* 2025.

[7] A. Das et al., "IoT-Based Smart Monitoring System with AI Decision Support Using ESP32," *International Journal of IoT Applications,* 2025.

[8] Vinit et al., "IoT-Based Monitoring System Using ESP32 for Real-Time Applications," *IEEE TEMSMET Conference,* 2025.

[9] M. Abdullah et al., "Cellular IoT-Based Environmental Monitoring System Using ESP32," *MDPI Engineering Proceedings,* 2025.

[10] D. Vaithiyanathan et al., "ESP32-Based Industrial Monitoring and Alert System with Cloud Integration," *International Conference on Smart Systems,* 2024.

[11] J. Lee, B. Bagheri & H. Kao, "A Cyber-Physical Systems Architecture for Industry 4.0 Manufacturing," *Manufacturing Letters,* 2022.

[12] P. Singh and K. Mehta, "Web-Based Dashboard for IoT Monitoring Using HTML, CSS, and JavaScript," *International Journal of Computer Applications,* 2023.

[13] Espressif Systems, "ESP32 Technical Reference Manual," ESP32 DevKit V1, 2024.

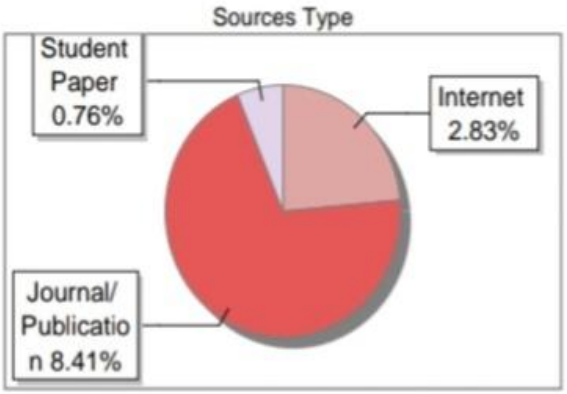
[14] Google, "Firebase Realtime Database Documentation," Firebase Realtime Database, 2024.

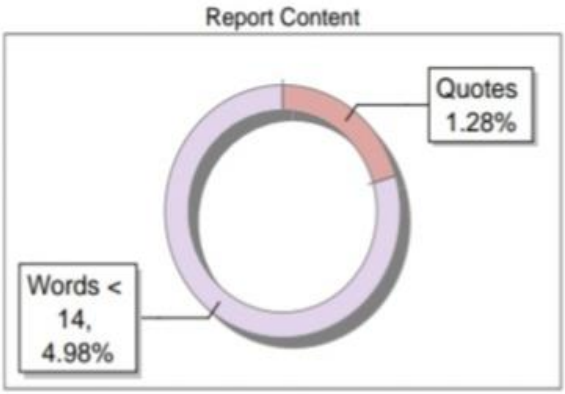
[15] D. Witczak, "Review of Monitoring and Control Systems Based on IoT," *Journal of Industrial Information Systems,* 2024.

Plagiarism Report

Result Information

Similarity **12 %**





Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

Research Paper Publication Certificate

International Research Journal Of Modernization in Engineering Technology and Science

(Peer-Reviewed, Open Access, Fully Refereed International Journal)

e-ISSN: 2582-5208

Ref: IRJMETS/Certificate/Volume 08/Issue 04/ 80400279768

DOI: https://www.doi.org/10.56726/IRJMETS95600

Date: 25/04/2026

Certificate of Publication

*This is to certify that author "**Sumedh Sanjay Varadkar**" with paper ID "**IRJMETS80400279768**" has published a paper entitled "SMART FACTORY IOT DASHBOARD WITH EQUIPMENT CONTROL" **in International Research Journal Of Modernization In Engineering Technology And Science (IRJMETS), Volume 08, Issue 04, April 2026***

IRJMETS Impact Factor 8.187

We Wish For Your Better Future

www.irjmets.com



IRJMETS

International Research Journal Of Modernization in Engineering Technology and Science

(Peer-Reviewed, Open Access, Fully Refereed International Journal)

e-ISSN: 2582-5208

Ref: IRJMETS/Certificate/Volume 08/Issue 04/ 80400279768

DOI: https://www.doi.org/10.56726/IRJMETS95600

Date: 25/04/2026

Certificate of Publication

*This is to certify that author "**Shubham Rajan Malvankar**" with paper ID "**IRJMETS80400279768**" has published a paper entitled "SMART FACTORY IOT DASHBOARD WITH EQUIPMENT CONTROL" **in International Research Journal Of Modernization In Engineering Technology And Science (IRJMETS), Volume 08, Issue 04, April 2026***

IRJMETS Impact Factor 8.187

We Wish For Your Better Future

www.irjmets.com

Google scholar · issuu · Academia.edu · Mendeley Advisor Community · doi · Crossref Content Registration



International Research Journal Of Modernization in Engineering Technology and Science

(Peer-Reviewed, Open Access, Fully Refereed International Journal)

e-ISSN: 2582-5208

Ref: IRJMETS/Certificate/Volume 08/Issue 04/ 80400279768

DOI: https://www.doi.org/10.56726/IRJMETS95600

Date: 25/04/2026

Certificate of Publication

*This is to certify that author "**Shreyash Babaso Patil**" with paper ID "**IRJMETS80400279768**" has published a paper entitled "SMART FACTORY IOT DASHBOARD WITH EQUIPMENT CONTROL" **in International Research Journal Of Modernization In Engineering Technology And Science (IRJMETS), Volume 08, Issue 04, April 2026***

IRJMETS Impact Factor 8.187

We Wish For Your Better Future

www.irjmets.com



International Research Journal Of Modernization in Engineering Technology and Science

(Peer-Reviewed, Open Access, Fully Refereed International Journal)

e-ISSN: 2582-5208

Ref: IRJMETS/Certificate/Volume 08/Issue 04/ 80400279768

DOI: https://www.doi.org/10.56726/IRJMETS95600

Date: 25/04/2026

Certificate of Publication

*This is to certify that author "**Sanket Tatyaso Balanna**" with paper ID "**IRJMETS80400279768**" has published a paper entitled "SMART FACTORY IOT DASHBOARD WITH EQUIPMENT CONTROL" **in International Research Journal Of Modernization In Engineering Technology And Science (IRJMETS), Volume 08, Issue 04, April 2026***

IRJMETS Impact Factor 8.187

We Wish For Your Better Future

www.irjmets.com

International Research Journal Of Modernization in Engineering Technology and Science

(Peer-Reviewed, Open Access, Fully Refereed International Journal)

e-ISSN: 2582-5208

Ref: IRJMETS/Certificate/Volume 08/Issue 04/ 80400279768

DOI: https://www.doi.org/10.56726/IRJMETS95600

Date: 25/04/2026

Certificate of Publication

*This is to certify that author "**Vinayak. D. Chavan**" with paper ID "**IRJMETS80400279768**" has published a paper entitled "SMART FACTORY IOT DASHBOARD WITH EQUIPMENT CONTROL" **in International Research Journal Of Modernization In Engineering Technology And Science (IRJMETS), Volume 08, Issue 04, April 2026***

IRJMETS Impact Factor 8.187

We Wish For Your Better Future

www.irjmets.com